

Lab Report

Topic Name

Siddharth Narayan Mishra

123CS0189

CD Lab Assignment 11 Report

Objective

- To design and implement a parser using Lex and Yacc for recognizing BibTeX entries
 - To validate the structure of different BibTeX formats such as ARTICLE, BOOK, and INPROCEEDINGS
 - To convert valid BibTeX input into a structured XML representation
 - To detect and report both lexical and syntax errors with proper messages
-

Introduction

BibTeX is a standard format used for representing bibliographic references in a structured manner. Each entry in a BibTeX file follows a predefined format with fields such as author, title, year, etc. Since the structure is well-defined, it is suitable for processing using compiler design tools like Lex and Yacc.

In this experiment, we implement a parser that reads BibTeX entries, checks whether they follow the given grammar, and converts them into XML format. XML is chosen because it provides a hierarchical and machine-readable representation of data.

Methodology

The implementation is divided into two major phases: lexical analysis and syntax analysis.

Lexical Analysis

- Implemented using Lex (Flex)
 - The input file is scanned character by character to identify tokens
 - Tokens generated include:
 - Keywords: ARTICLE, BOOK, INPROCEEDINGS
 - Identifiers: TEXT, ALPHANUMERIC
 - Numbers: DIGIT
 - Special symbols: {, }, =, ,
 - Whitespace and irrelevant characters are ignored except newline and tab where required
 - Each recognized token is passed to the Yacc parser
-

Syntax Analysis

- Implemented using Yacc (Bison)
 - Grammar rules are defined based on the given specification
 - The start symbol handles multiple entries in sequence
 - Separate rules are defined for:
 - ARTICLE entries
 - BOOK entries
 - INPROCEEDINGS entries
 - Each block ensures that all required fields are present and in correct order
-

XML Generation

- Semantic actions are embedded within Yacc rules
 - As each rule is reduced, corresponding XML tags are printed
 - Mapping is done as follows:
 - Entry types → <ARTICLE>, <BOOK>, <INPROCEEDINGS>
 - Fields → corresponding XML tags such as <author>, <title>, etc.
 - The complete output is enclosed within a root <document> tag
 - This ensures the output is well-structured and readable
-

Error Handling

Error handling is an important part of the parser implementation.

- **Lexical Errors:**
 - Occur when invalid or unrecognized tokens are encountered
 - Reported with line number information
 - **Syntax Errors:**
 - Occur when the input does not follow the defined grammar
 - Examples include missing fields, incorrect order, or missing symbols
 - Handled using `yyerror()` function
 - The parser reports:
 - Type of error (lexical or syntax)
 - Position of error (line number)
 - The program terminates after detecting the first error to avoid cascading issues
-

Sample Input and Output

- The input consists of multiple BibTeX entries of different types
 - The output is a properly formatted XML document
 - Each entry is converted into its corresponding XML representation
 - All entries are grouped under a single `<document>` root element
-

Observations

- The use of grammar rules ensures strict validation of BibTeX structure
 - Lex simplifies token recognition, reducing complexity in parsing
 - Yacc efficiently handles hierarchical structure and rule enforcement
 - Embedding semantic actions allows simultaneous parsing and translation
-

Result

- The program successfully recognizes valid BibTeX entries
- Correct XML output is generated for each entry

- Errors are detected and reported accurately
 - Multiple entries are handled without issues
-

Conclusion

- A BibTeX parser was successfully implemented using Lex and Yacc
- The system validates input and converts it into structured XML format
- Error detection mechanisms improve robustness and correctness
- This experiment demonstrates the practical use of compiler design tools in real-world data processing